

Lab Exercise: Doubly Linked List in C

Learning Objectives

After completing this lab, students should be able to:

1. Understand the structure of a doubly linked list.
2. Create and traverse a doubly linked list.
3. Perform insertion and deletion operations.
4. Search for an element.
5. Understand the time complexity of each operation.

Lab Question

Write a C program to implement a Doubly Linked List with the following operations:

1. Create a doubly linked list.
2. Display the list from **left to right**.
3. Display the list from **right to left**.

4. Insert a node:

- At the beginning
- At the end
- At a given position

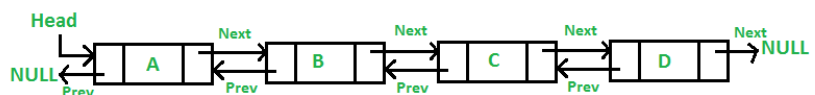
5. Delete a node:

- From the beginning
- From the end
- From a given position

6. Search for an element.

7. Display the number of nodes in the list.

8. Exit the program.



Also explain the algorithm and time complexity of each operation.

1. Structure of a Doubly Linked List

Unlike a singly linked list, every node contains **three fields**:

```
+-----+-----+-----+
| prev | data | next |
+-----+-----+-----+
```

For example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

- `prev` points to the previous node.
- `data` stores the value.
- `next` points to the next node.

The first node is called `head`, and the last node has `next = NULL`.

2. Complete C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;

/* Create a new node */
struct Node* createNode(int value) {
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = NULL;

    return newNode;
}

/* Insert at beginning */
void insertBeginning(int value) {
    struct Node *newNode = createNode(value);

    if (head == NULL) {
        head = newNode;
    } else {
```

```

        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    printf("%d inserted at beginning.\n", value);
}

/* Insert at end */
void insertEnd(int value) {
    struct Node *newNode = createNode(value);
    struct Node *temp;

    if (head == NULL) {
        head = newNode;
        printf("%d inserted at end.\n", value);
        return;
    }

    temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;

    printf("%d inserted at end.\n", value);
}

/* Insert at a given position */
void insertPosition(int value, int position) {
    struct Node *newNode;
    struct Node *temp;
    int i;

    if (position <= 0) {
        printf("Invalid position.\n");
        return;
    }

    if (position == 1) {
        insertBeginning(value);
        return;
    }

    if (head == NULL) {
        printf("List is empty. Only position 1 is valid.\n");
        return;
    }

```

```

temp = head;

for (i = 1; i < position - 1 && temp != NULL; i++) {
    temp = temp->next;
}

if (temp == NULL) {
    printf("Invalid position.\n");
    return;
}

newNode = createNode(value);

newNode->next = temp->next;
newNode->prev = temp;

if (temp->next != NULL) {
    temp->next->prev = newNode;
}

temp->next = newNode;

printf("%d inserted at position %d.\n", value, position);
}

/* Delete from beginning */
void deleteBeginning() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    head = head->next;

    if (head != NULL) {
        head->prev = NULL;
    }

    printf("%d deleted from beginning.\n", temp->data);

    free(temp);
}

/* Delete from end */
void deleteEnd() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");

```

```

        return;
    }

    temp = head;

    if (temp->next == NULL) {
        printf("%d deleted from end.\n", temp->data);
        free(temp);
        head = NULL;
        return;
    }

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->prev->next = NULL;

    printf("%d deleted from end.\n", temp->data);

    free(temp);
}

/* Delete from a given position */
void deletePosition(int position) {
    struct Node *temp;
    int i;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    if (position <= 0) {
        printf("Invalid position.\n");
        return;
    }

    if (position == 1) {
        deleteBeginning();
        return;
    }

    temp = head;

    for (i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position.\n");
        return;
    }

```

```

    }

    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }

    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }

    printf("%d deleted from position %d.\n",
           temp->data, position);

    free(temp);
}

/* Display from beginning to end */
void displayForward() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    printf("Forward: ");

    while (temp != NULL) {
        printf("%d", temp->data);

        if (temp->next != NULL) {
            printf(" <-> ");
        }

        temp = temp->next;
    }

    printf("\n");
}

/* Display from end to beginning */
void displayBackward() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;

```

```

        while (temp->next != NULL) {
            temp = temp->next;
        }

        printf("Backward: ");

        while (temp != NULL) {
            printf("%d", temp->data);

            if (temp->prev != NULL) {
                printf(" <-> ");
            }

            temp = temp->prev;
        }

        printf("\n");
    }

    /* Search an element */
    void search(int value) {
        struct Node *temp = head;
        int position = 1;

        while (temp != NULL) {
            if (temp->data == value) {
                printf("%d found at position %d.\n",
                    value, position);
                return;
            }

            temp = temp->next;
            position++;
        }

        printf("%d not found in the list.\n", value);
    }

    /* Count number of nodes */
    void countNodes() {
        struct Node *temp = head;
        int count = 0;

        while (temp != NULL) {
            count++;
            temp = temp->next;
        }

        printf("Number of nodes = %d\n", count);
    }

```

```

/* Main function */
int main() {
    int choice;
    int value;
    int position;

    while (1) {

        printf("\n===== DOUBLY LINKED LIST =====\n");
        printf("1. Insert at beginning\n");
        printf("2. Insert at end\n");
        printf("3. Insert at position\n");
        printf("4. Delete from beginning\n");
        printf("5. Delete from end\n");
        printf("6. Delete from position\n");
        printf("7. Display forward\n");
        printf("8. Display backward\n");
        printf("9. Search\n");
        printf("10. Count nodes\n");
        printf("11. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertBeginning(value);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertEnd(value);
                break;

            case 3:
                printf("Enter value: ");
                scanf("%d", &value);

                printf("Enter position: ");
                scanf("%d", &position);

                insertPosition(value, position);
                break;

            case 4:
                deleteBeginning();
                break;

```

```

        case 5:
            deleteEnd();
            break;

        case 6:
            printf("Enter position: ");
            scanf("%d", &position);

            deletePosition(position);
            break;

        case 7:
            displayForward();
            break;

        case 8:
            displayBackward();
            break;

        case 9:
            printf("Enter value to search: ");
            scanf("%d", &value);

            search(value);
            break;

        case 10:
            countNodes();
            break;

        case 11:
            printf("Program terminated.\n");
            exit(0);

        default:
            printf("Invalid choice!\n");
    }
}

```

3. Important Operations Explained

A. Insertion at Beginning

Suppose the list is:

```
NULL <- 20 <-> 30 <-> 40 -> NULL
```

Insert 10 .

We create:

10

Then:

C

```
newNode->next = head;  
head->prev = newNode;  
head = newNode;
```

Result:

NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL

Complexity

Time: $O(1)$

Because we don't need to traverse the list.

4. Insertion at End

Initially:

10 <-> 20 <-> 30

To insert 40 , we traverse until:

```
temp->next == NULL
```

Then:

C

```
temp->next = newNode;  
newNode->prev = temp;
```

Result:

10 <-> 20 <-> 30 <-> 40

Complexity

Time: $O(n)$

Because we traverse the list to find the last node.

If a `tail` pointer is maintained, insertion at the end can be performed in $O(1)$.

5. Insertion at a Given Position

Suppose:

10 <-> 20 <-> 40

Insert 30 at position 3.

Before:

10 <-> 20 <-> 40

After:

10 <-> 20 <-> 30 <-> 40

The important pointer changes are:

C

```
newNode->next = temp->next;  
newNode->prev = temp;  
  
temp->next->prev = newNode;  
temp->next = newNode;
```

There are **four pointer relationships** to maintain.

Complexity

Time: $O(n)$

Because we may have to traverse to the required position.

6. Deletion from Beginning

Suppose:

```
10 <-> 20 <-> 30
```

Delete 10 .

We move head :

C

```
head = head->next;
```

Then make the new head's previous pointer NULL :

C

```
head->prev = NULL;
```

Result:

```
NULL <- 20 <-> 30 -> NULL
```

Finally:

C

```
free(temp);
```

Complexity

Time: $O(1)$

7. Deletion from End

Suppose:

```
10 <-> 20 <-> 30
```

We move to the last node:

```
temp = 30
```

Since 30 is the last node:

C

```
temp->prev->next = NULL;
```

Result:

```
10 <-> 20 -> NULL
```

Then:

C

```
free(temp);
```

Complexity

Time: $O(n)$

Because we traverse the list to find the last node.

Again, with a `tail` pointer, deletion from the end can be **$O(1)$** .

8. Deletion at a Given Position

Suppose:

```
10 <-> 20 <-> 30 <-> 40
```

Delete position 3.

Node `30` has:

```
prev = 20  
next = 40
```

We connect:

```
20 <-> 40
```

using:

C

```
temp->next->prev = temp->prev;
temp->prev->next = temp->next;
```

Then:

C

```
free(temp);
```

Result:

10 <-> 20 <-> 40

Complexity

Time: $O(n)$

9. Forward Traversal

Forward traversal starts from `head`.

C

```
temp = head;

while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
```

For:

10 <-> 20 <-> 30 <-> 40

Output:

10 20 30 40

Complexity

Time: $O(n)$

10. Backward Traversal

This is one of the important advantages of a doubly linked list.

First reach the last node:

C

```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

Then use `prev`:

C

```
while (temp != NULL) {  
    printf("%d ", temp->data);  
    temp = temp->prev;  
}
```

Output:

40 30 20 10

Complexity

Time: `O (n)`

11. Searching

For searching `30`:

10 <-> 20 <-> 30 <-> 40

Start from the first node and compare:

10 → 20 → 30 ✓

Complexity

Case	Complexity
Best case	<code>O (1)</code>
Average case	<code>O (n)</code>

Case

Complexity

Worst case

$O(n)$

12. Algorithmic Analysis

Let n be the number of nodes.

Operation	Time Complexity	Reason
Create node	$O(1)$	Allocate one node
Insert beginning	$O(1)$	Directly modify <code>head</code>
Insert end	$O(n)$	Traverse to last node
Insert at position	$O(n)$	Traverse to position
Delete beginning	$O(1)$	Directly modify <code>head</code>
Delete end	$O(n)$	Traverse to last node
Delete at position	$O(n)$	Traverse to position
Forward traversal	$O(n)$	Visit every node
Backward traversal	$O(n)$	Reach end + visit nodes
Search	$O(n)$	Sequential search
Count nodes	$O(n)$	Visit every node

Space Complexity

For n nodes:

Space = $O(n)$

Each node requires memory for:

```
data
prev pointer
next pointer
```

Therefore, a doubly linked list requires **more memory than a singly linked list**.

13. Sample Execution

```
===== DOUBLY LINKED LIST =====
```

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete from position
7. Display forward
8. Display backward
9. Search
10. Count nodes
11. Exit

```
Enter your choice: 1
Enter value: 10
10 inserted at beginning.
```

```
Enter your choice: 2
Enter value: 20
20 inserted at end.
```

```
Enter your choice: 2
Enter value: 30
30 inserted at end.
```

```
Enter your choice: 7
Forward: 10 <-> 20 <-> 30
```

```
Enter your choice: 8
Backward: 30 <-> 20 <-> 10
```

```
Enter your choice: 3
Enter value: 25
Enter position: 3
25 inserted at position 3.
```

```
Enter your choice: 7
Forward: 10 <-> 20 <-> 25 <-> 30
```

```
Enter your choice: 9
Enter value to search: 25
25 found at position 3.
```

```
Enter your choice: 6
Enter position: 3
```

25 deleted from position 3.

Enter your choice: 7

Forward: 10 <-> 20 <-> 30

Lab Questions for Students

Question 1 — Basic

Modify the program to implement a function:

C

```
void insertAfterValue(int key, int value);
```

For example:

10 <-> 20 <-> 30

Insert 25 after 20 :

10 <-> 20 <-> 25 <-> 30

Question 2 — Deletion

Implement:

C

```
void deleteValue(int value);
```

The function should delete the **first occurrence** of the specified value.

Question 3 — Algorithm Analysis

For every function in the program, identify:

- Best-case complexity
- Worst-case complexity
- Space complexity

Question 4 — Conceptual

Explain why a doubly linked list requires more memory than a singly linked list.

Question 5 — Advanced

Modify the program to maintain both:

C

```
struct Node *head;  
struct Node *tail;
```

Then implement insertion and deletion at both ends in **$O(1)$** time.